

Long short Term Memory (LSTM)

Natural Language Processing

LSTM Model

Long Short-Term Memory (LSTM) cell — a special kind of Recurrent Neural Network (RNN) architecture designed to better handle long-term dependencies in sequence data.

OR

An **LSTM model**, or **Long Short-Term Memory model**, is a type of **Recurrent Neural Network (RNN)** designed to learn from **sequential data** while avoiding some key limitations of traditional RNNs, such as the **vanishing gradient problem**.

LSTM: How it works

An LSTM unit consists of a **cell state** and three main **gates** that regulate the flow of information:

- **Forget Gate:** Decides what information from the previous cell state to forget.
- **Input Gate:** Determines what new information to add to the cell state.
- **Output Gate:** Controls what part of the cell state to output as the hidden state.

These gates use **sigmoid** and **tanh** activation functions to decide how much information to keep or discard.



Neural Network
Layer



Pointwise
Operation



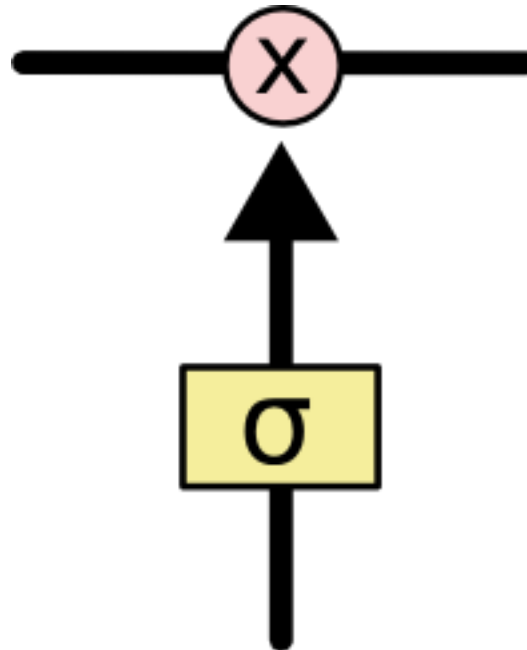
Vector
Transfer



Concatenate

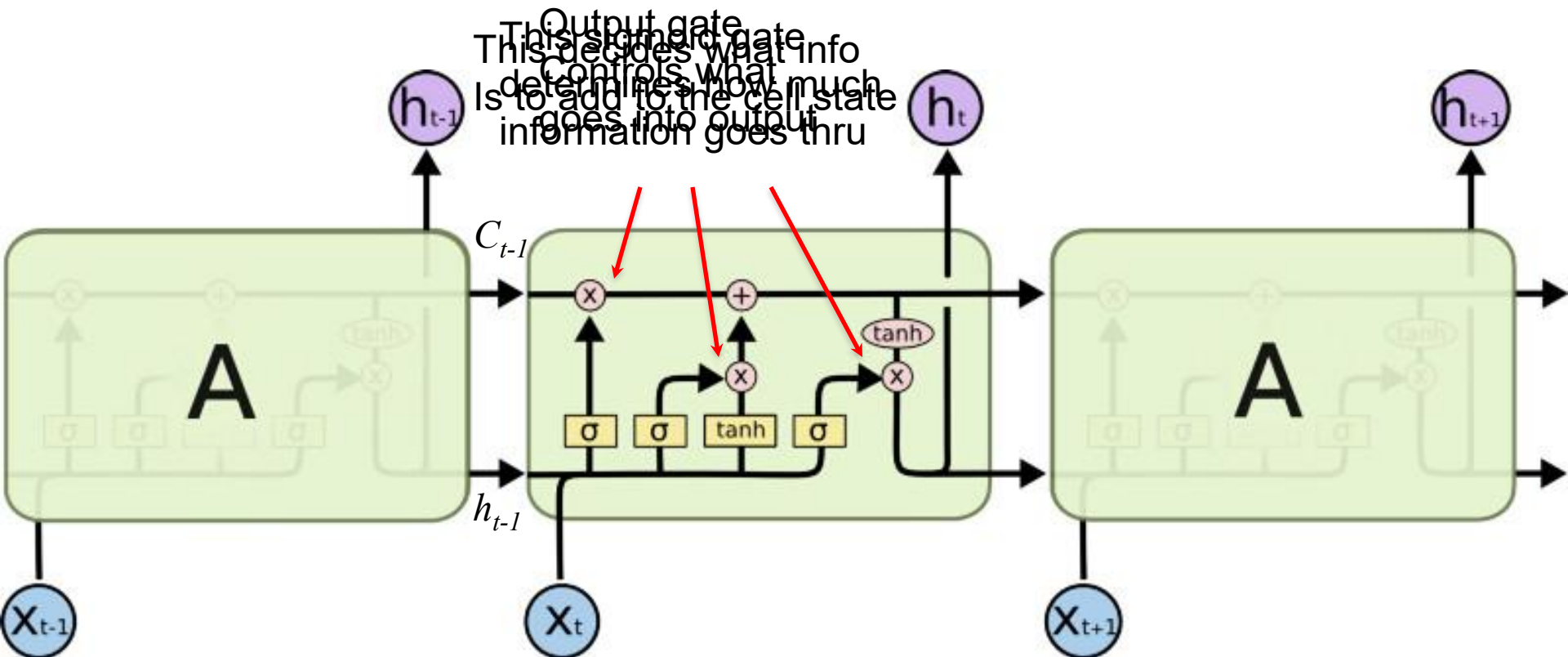


Copy



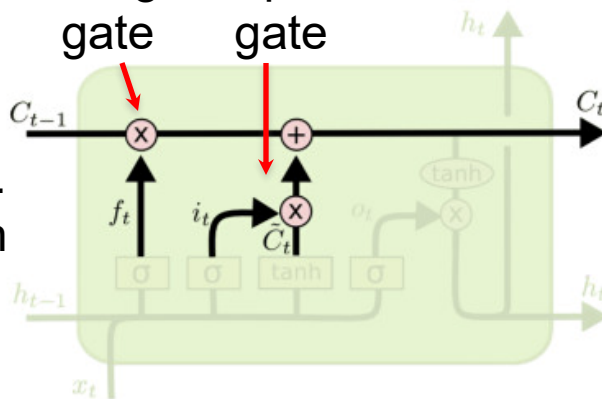
The sigmoid layer outputs numbers between 0-1 determine how much each component should be let through. Pink X gate is point-wise multiplication.

LSTM



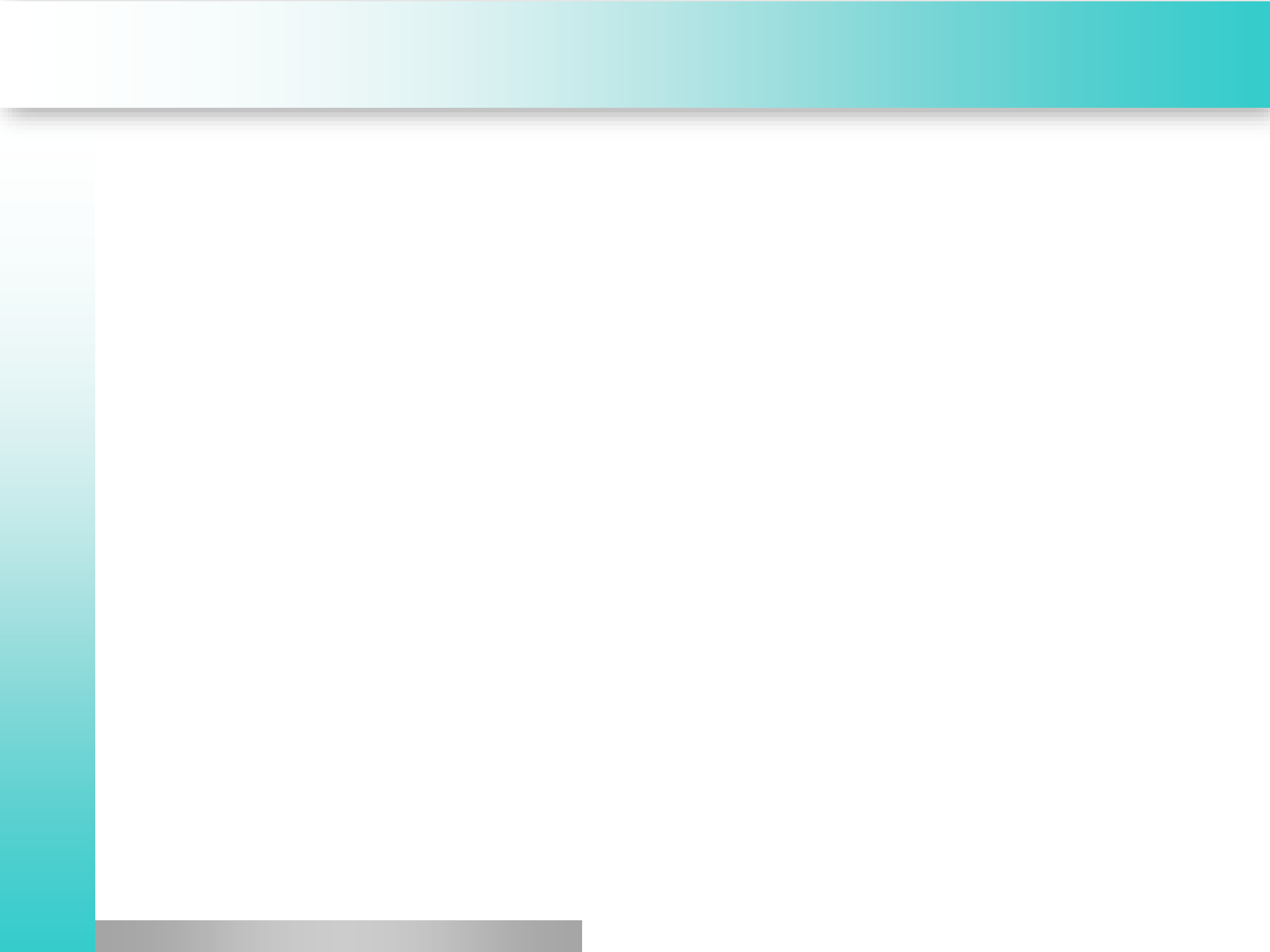
Output gate
This decides what info
controls what
determines how much
Is to add to the cell state
information goes thru

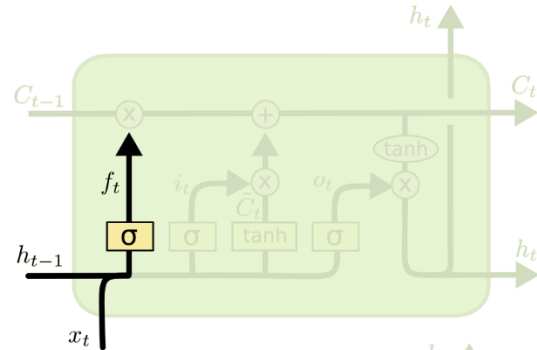
Forget input
gate gate



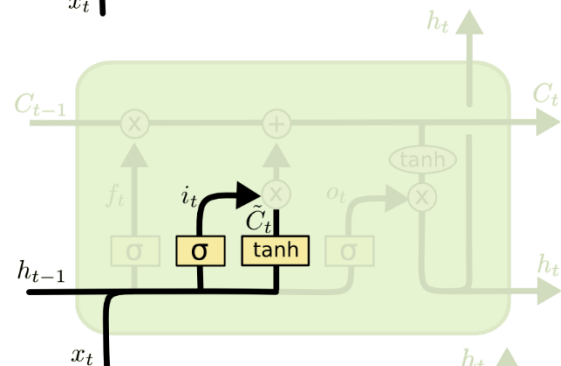
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

The core idea is this cell
state C , it is changed
Why sigmoid or tanh:
Sigmoid: 0, 1 gating as switch.
slowly, with only minor
Vanishing gradient problem in
linear interactions. It is very
LSTM is handled already.
easy for information to flow
ReLU replaces tanh ok?
along it unchanged.



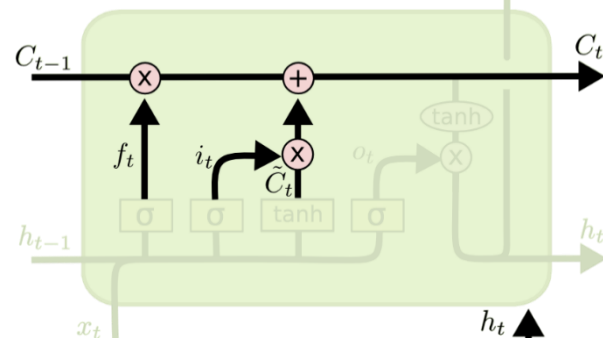


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

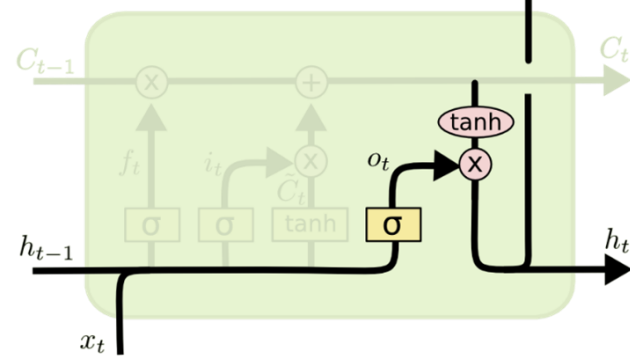


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



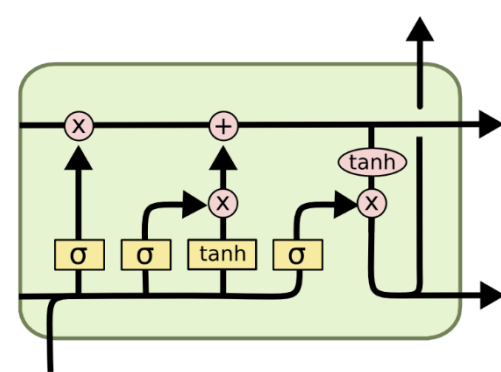
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Decide what part of the cell state to output

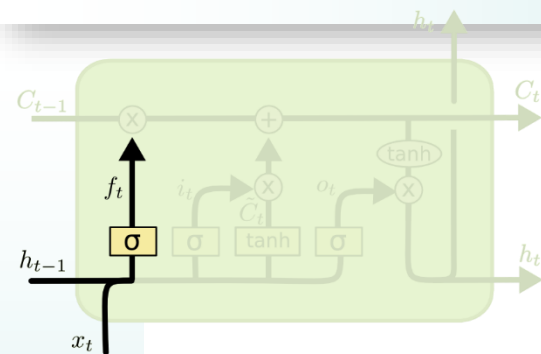


i_t decides what component is to be updated.

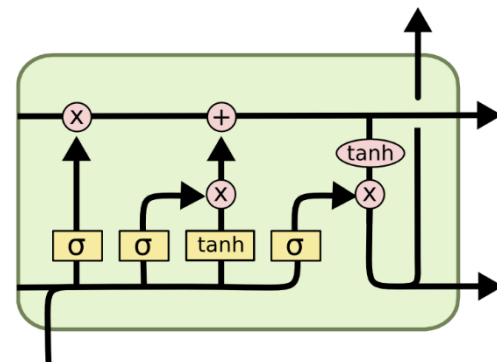
C'_t provides change contents

Updating the cell state

LSTM Model: 1. Forget Gate



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$



Decides what portion of the previous cell state C_{t-1} to **keep** (remember) or **discard** (forget). So: If an element of f_t is 0 $\rightarrow$ forget that part of the state and If it's 1 $\rightarrow$ keep it entirely.

Symbol

Meaning

f_t

Forget gate output (a vector of values between 0 and 1)

σ

Sigmoid activation function — maps input to [0, 1]

W_f

Weight matrix for the forget gate

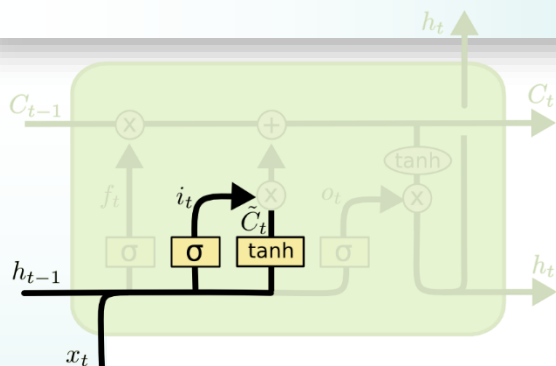
$[h_{t-1}, x_t]$

Concatenation of previous hidden state and current input

b_f

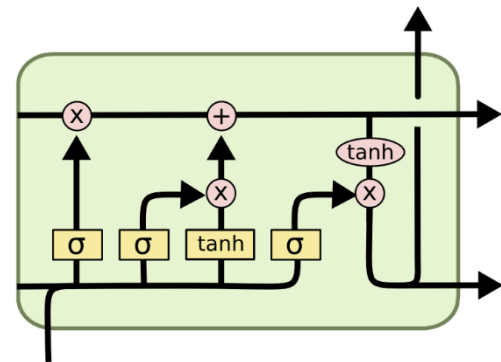
Bias vector for the forget gate

LSTM Model: 2. Input Gate



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

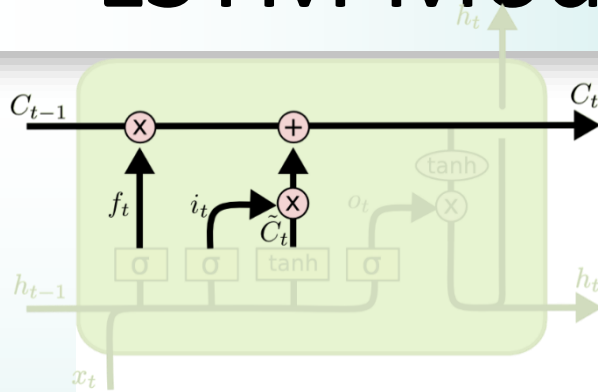


i_t : Decides which parts of the new information to **add** to the cell state.

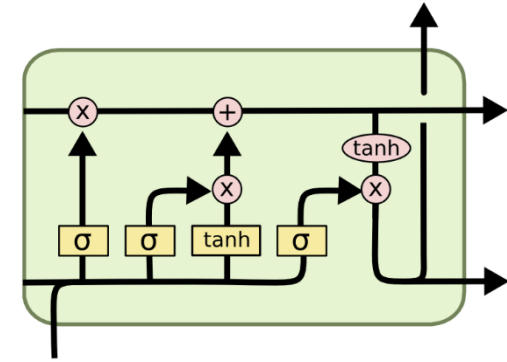
C_t : Contains **new candidate values** that could be added to the cell state.

Symbol	Meaning
i_t	Input gate output (between 0 and 1)
W_i	Weight matrix for the input gate
b_i	Bias for the input gate
$\tilde{C}_t$	Candidate cell state update (range: -1 to 1)
W_C	Weight matrix for candidate cell update
b_C	Bias for candidate update
$\tanh$	Hyperbolic tangent activation function (outputs between -1 and 1)

LSTM Model: 3. Update Cell State



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



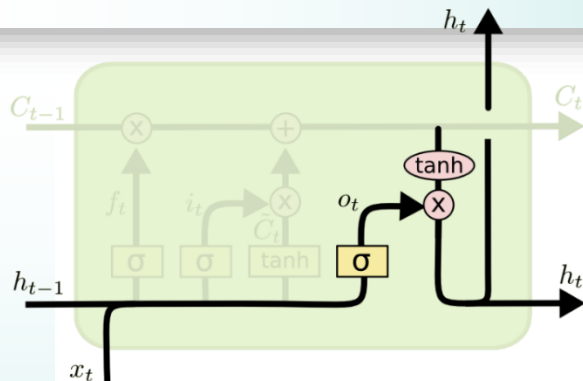
$f_t * C_{t-1}$: What to **keep** from old cell state

$i_t * \tilde{C}_t$: What to **add** from the new candidate

So, this equation **updates the cell's internal memory**.

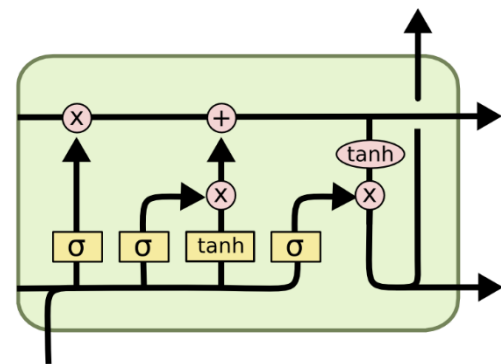
Symbol	Meaning
C_t	New cell state (memory at current time step)
$*$	Element-wise multiplication
$f_t \cdot C_{t-1}$	Old memory after forget filtering
$i_t \cdot \tilde{C}_t$	New information to add

LSTM Model: 4. Output Gate



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$



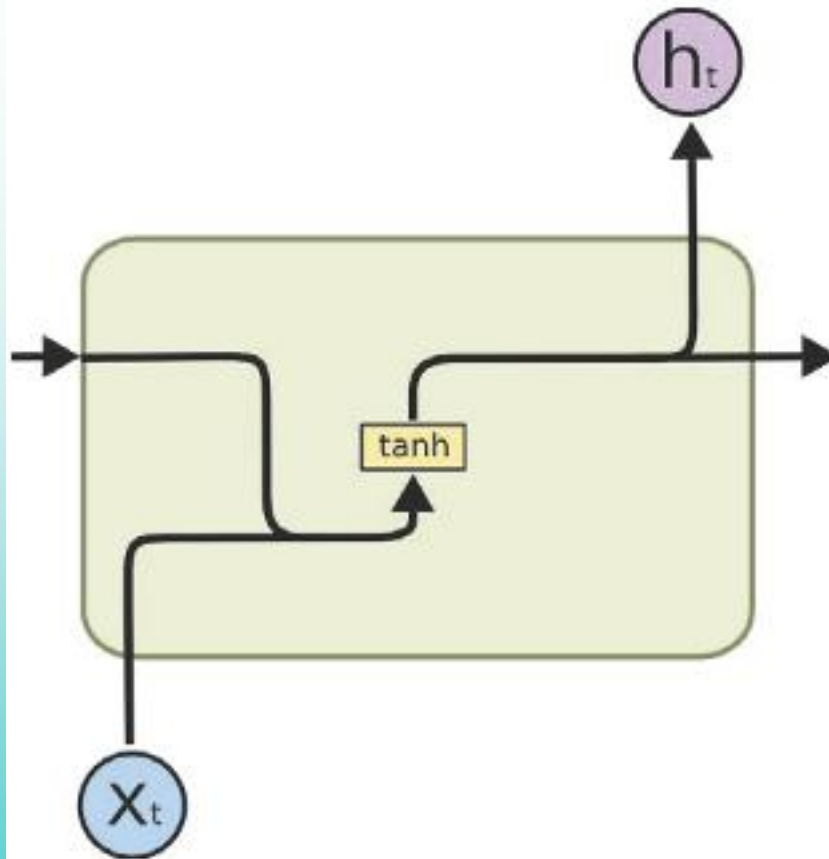
O_t : Decides **what part of the cell state to output**.

h_t : Final hidden state (also becomes output and input for the next timestep).

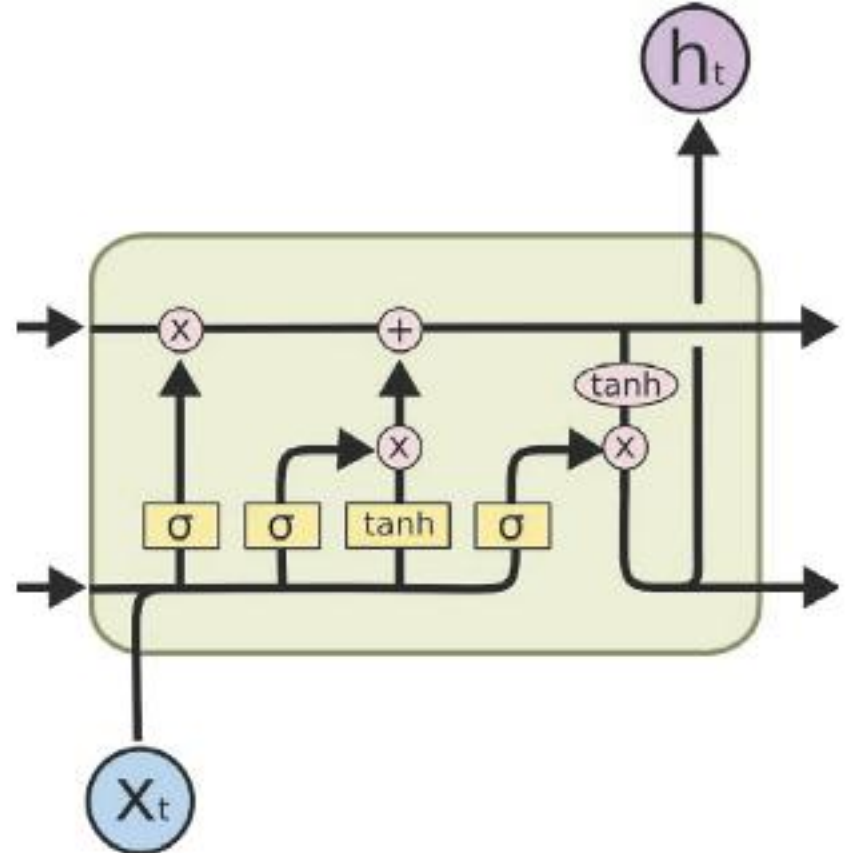
Symbol	Meaning
o_t	Output gate vector (between 0 and 1)
W_o, b_o	Weight and bias for output gate
h_t	Final output/hidden state from this timestep
C_t	Updated cell state
$\tanh(C_t)$	Scaled version of cell state to be output
*	Element-wise multiplication

So, h_t is a **filtered version of the memory**, based on how much to reveal (controlled by o_t)

RNN vs LSTM

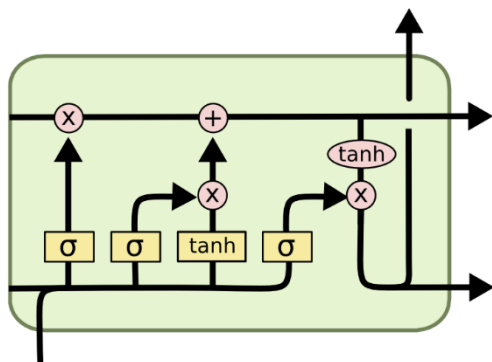


(a) RNN



(b) LSTM

Information Flow in a LSTM



c^{t-1}

These 4 matrix computation can be done concurrently.

Controls forget gate

Controls input gate

Updating information

Controls Output gate

z^f

z^i

z

z^o



h^{t-1}

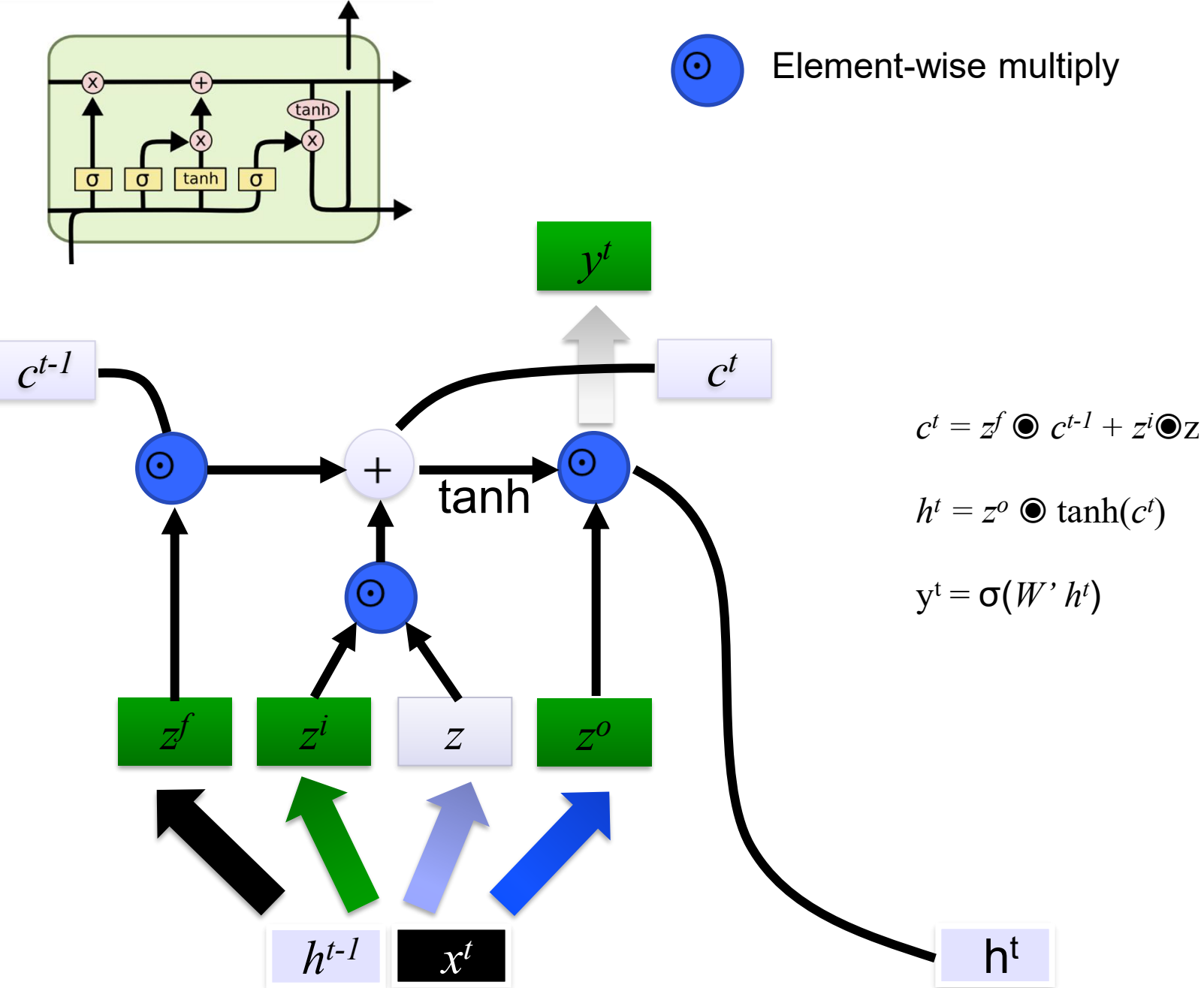
x^t

$$z = \tanh(W \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix})$$

$$z^i = \sigma(W^i \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix})$$

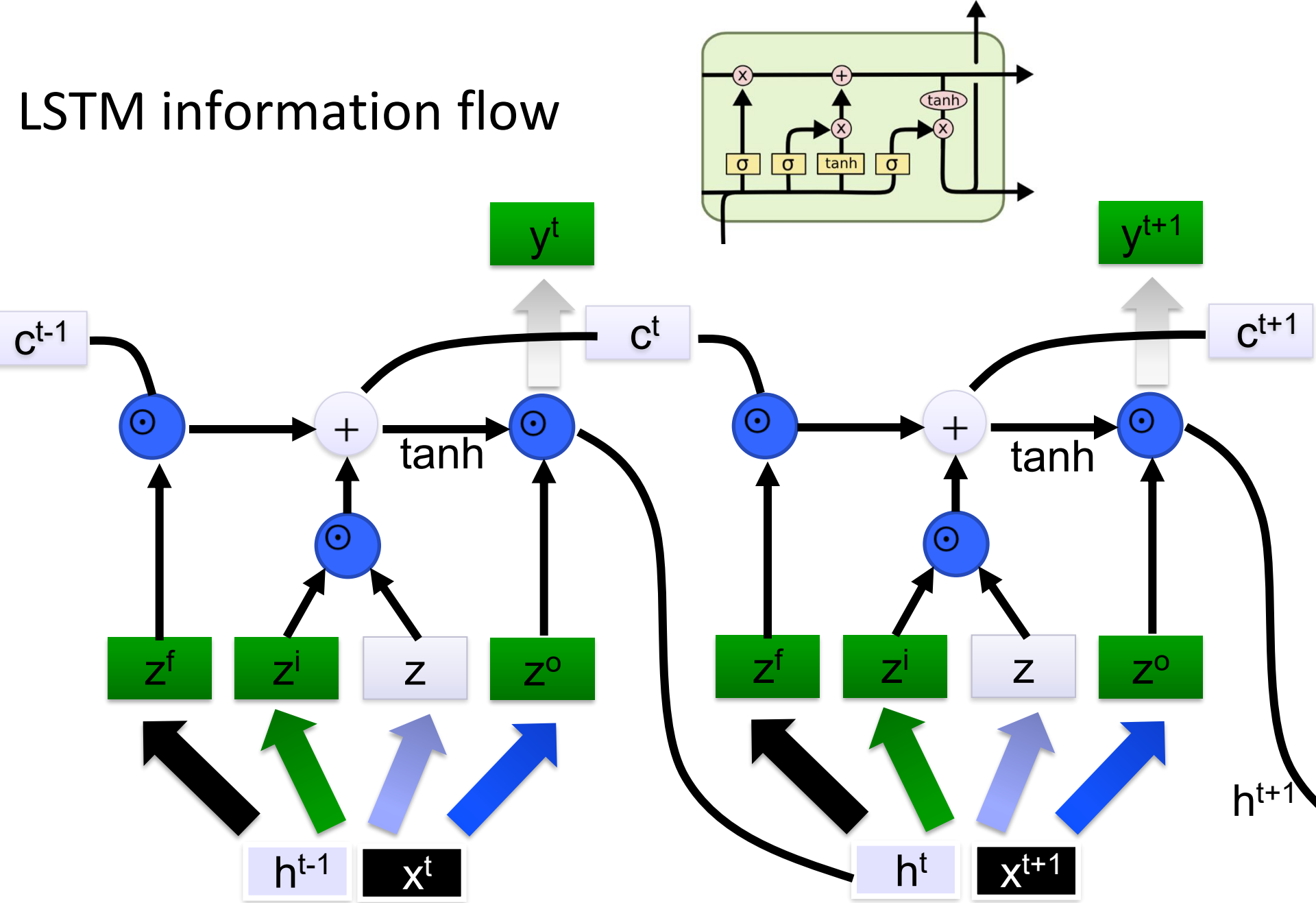
$$z^f = \sigma(W^f \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix})$$

$$z^o = \sigma(W^o \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix})$$



Information flow of LSTM

LSTM information flow



Gated Recurrent Unit (GRU)

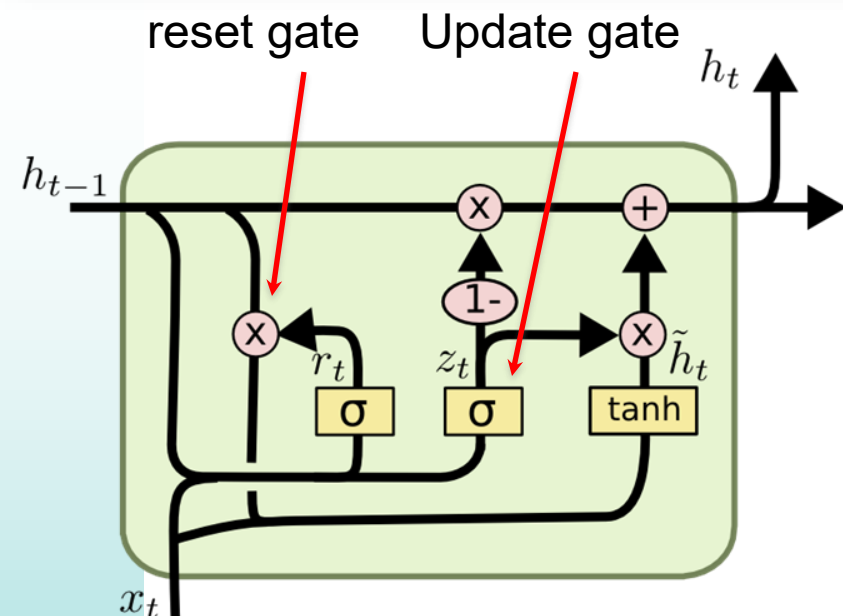
Gated Recurrent Unit (GRU) — a simplified version of the LSTM.

GRU combines the memory cell and hidden state into a single state vector h_t , and simplifies the gating mechanism. It uses only **two gates**:

- 1.Update Gate** z_t : – controls how much of the past information to carry forward.
- 2.Reset Gate** r_t : – controls how much of the past information to forget.

It combines the **forget** and **input** into a single **update gate**. It also merges the cell state and hidden state. This is simpler than LSTM. There are many other variants too.

GRU – Gated Recurrent Unit



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

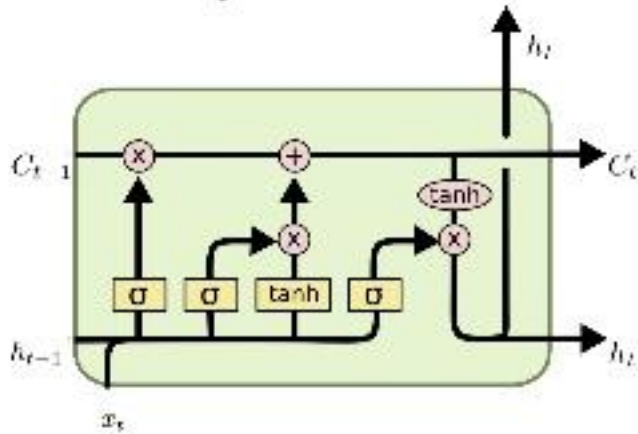
$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

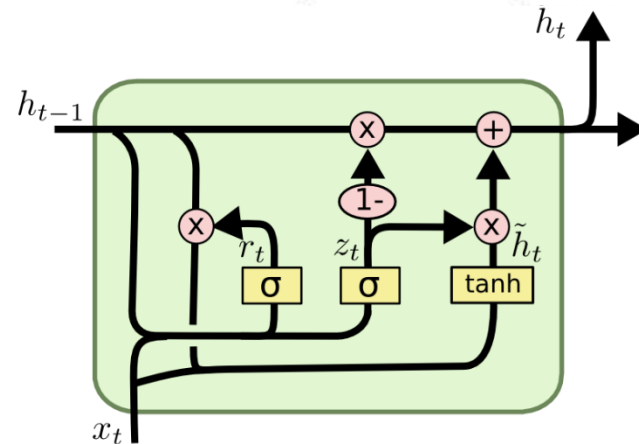
Component	Function
z_t (update gate)	Controls what to keep from the past
r_t (reset gate)	Controls what to forget from the past
$\tilde{h}_t$	Proposed new state
h_t	Final hidden state (output and memory)

LSTM and GRU

- LSTM [Hochreiter&Schmidhuber97]



- GRU [Cho+14]



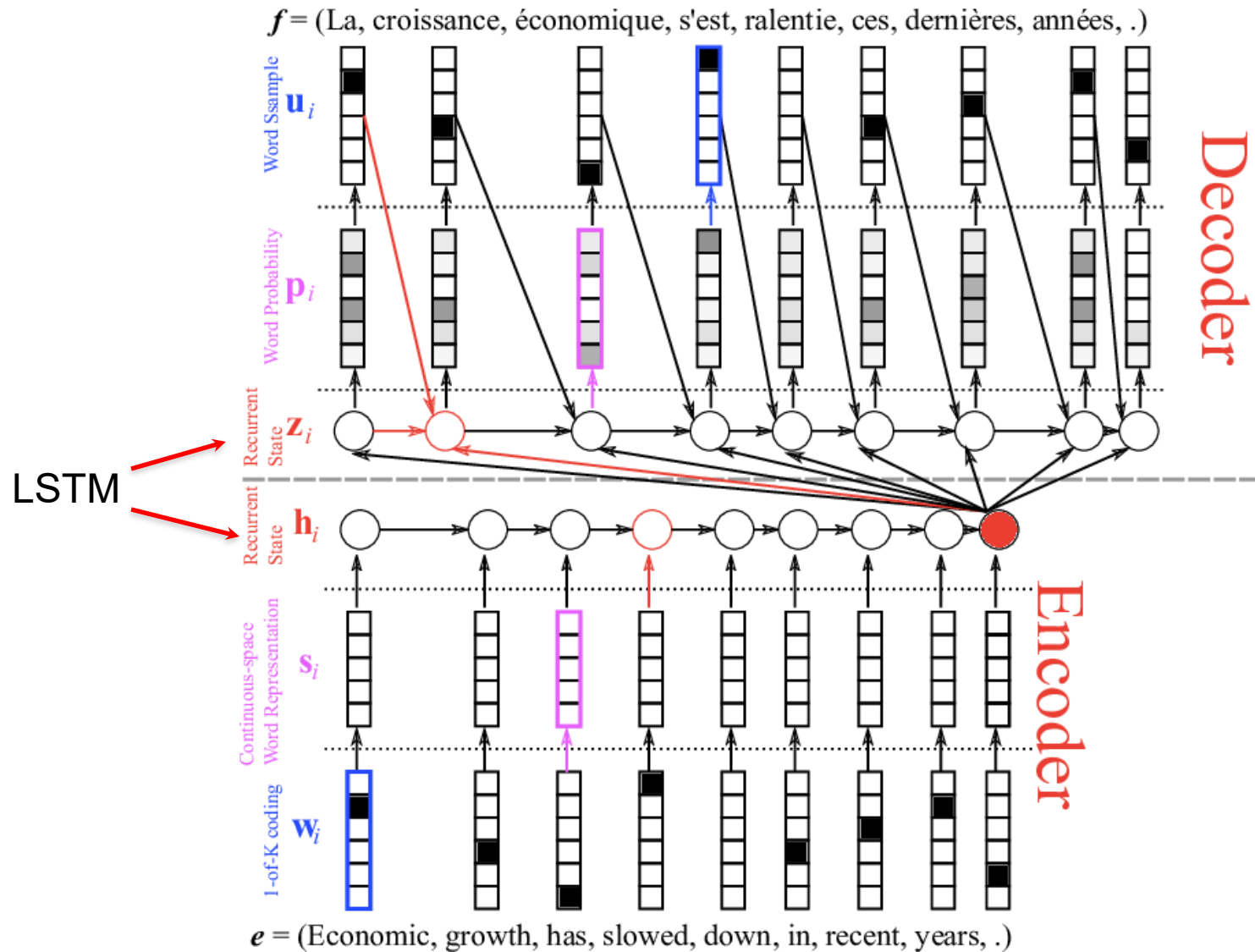
Both take x_t and h_{t-1} as inputs and compute h_t to pass along. GRUs don't need the cell layer to pass values along. The calculations within each iteration ensure that the h_t values being passed along either retain a high amount of old information or are jump-started with a high amount of new information.

Summary of LSTM and GRU

Feature	LSTM	GRU
Gates	3: Input, Forget, Output	2: Update, Reset
Cell state	Maintains a separate cell state C_t	No separate cell state — only hidden state h_t
Complexity	More complex (more parameters, more control)	Simpler (fewer gates, faster to train)
Output equation	$h_t = o_t * \tanh(C_t)$	$h_t = (1 - z_t) * \tilde{h}_t + z_t * h_{t-1}$
Performance	Better for learning long-term dependencies	Faster and often performs similarly on smaller datasets
Memory control	Explicit memory cell update	Implicit memory control through hidden state

Applications of LSTM / RNN

Neural machine translation



Sequence to sequence chat model

